

evaluators

Evaluators

The UserFlowPlugin registers 12 assertion evaluators with the challenge framework's assertion engine. Each evaluator implements the `assertion.Evaluator` function signature:

```
type Evaluator func(def assertion.Definition, value any) (bool,  
    string)
```

- `def` is the assertion definition, which includes a `Value` field for expected values.
- `value` is the actual value to evaluate.
- Returns (passed `bool`, message `string`).

Registration

All evaluators are registered via `RegisterEvaluators()`, called during `UserFlowPlugin.Init()`:

```
func RegisterEvaluators(engine *assertion.DefaultEngine) error
```

This registers each evaluator by name. The evaluator name is used as the assertion type in challenge definitions.

Evaluator Reference

build_succeeds

Checks that a build operation succeeded.

- **Input type:** `bool`
- **Pass condition:** value is `true`
- **def.Value:** not used
- **Pass message:** "build succeeded"
- **Fail message:** "build failed"

all_tests_pass

Checks that no test failures occurred.

- **Input type:** int (failure count)
- **Pass condition:** value equals 0
- **def.Value:** not used
- **Pass message:** "all tests passed (0 failures)"
- **Fail message:** "tests failed: N failures"

Accepts int, int64, float64, and float32 via the toIntVal helper.

lint_passes

Checks that a lint operation passed.

- **Input type:** bool
- **Pass condition:** value is true
- **def.Value:** not used
- **Pass message:** "lint passed"
- **Fail message:** "lint failed"

app_launches

Checks that an application launched successfully.

- **Input type:** bool
- **Pass condition:** value is true
- **def.Value:** not used
- **Pass message:** "app launched successfully"
- **Fail message:** "app failed to launch"

app_stable

Checks that an application remained stable (did not crash).

- **Input type:** bool
- **Pass condition:** value is true
- **def.Value:** not used
- **Pass message:** "app is stable"
- **Fail message:** "app is unstable"

status_code

Checks that an HTTP status code matches the expected value.

- **Input type:** int (actual status code)
- **Pass condition:** value equals def.Value
- **def.Value:** int (expected status code)
- **Pass message:** "status code is N"

- **Fail message:** "status code: expected N, got M"

Both actual and expected values are converted via `toIntVal`, accepting `int`, `int64`, `float64`, `float32`.

response_contains

Checks that a response string contains an expected substring.

- **Input type:** string (response body or text)
- **Pass condition:** value contains `def.Value`
- **def.Value:** string (expected substring)
- **Pass message:** "response contains \"X\""
 - **Fail message:** "response does not contain \"X\""

Uses `strings.Contains` for the comparison.

response_not_empty

Checks that a response has non-zero length.

- **Input type:** string or `[]byte`
- **Pass condition:** `len(value) > 0`
- **def.Value:** not used
- **Pass message:** "response is not empty"
- **Fail message:** "response is empty"

json_field_equals

Checks that a JSON field value equals the expected value.

- **Input type:** any (the field value)
- **Pass condition:** `fmt.Sprintf("%v", value) == fmt.Sprintf("%v", def.Value)`
- **def.Value:** any (expected value)
- **Pass message:** "field equals \"X\""
 - **Fail message:** "field: expected \"X\", got \"Y\""

Uses string-based comparison via `fmt.Sprintf`, which handles cross-type comparisons (e.g., `int` vs `float64`).

screenshot_exists

Checks that a screenshot was captured (non-empty byte slice).

- **Input type:** `[]byte` (screenshot data)
- **Pass condition:** `len(value) > 0`
- **def.Value:** not used
- **Pass message:** "screenshot captured"

- **Fail message:** "screenshot is empty"

flow_completes

Checks that an entire flow completed successfully.

- **Input type:** bool
- **Pass condition:** value is true
- **def.Value:** not used
- **Pass message:** "flow completed successfully"
- **Fail message:** "flow did not complete"

within_duration

Checks that an operation completed within a time limit.

- **Input type:** int (actual duration in milliseconds)
- **Pass condition:** value <= def.Value
- **def.Value:** int (maximum duration in milliseconds)
- **Pass message:** "duration Nms within limit Mms"
- **Fail message:** "duration Nms exceeds limit Mms"

Both values are converted via toIntVal.

Usage Example

After the plugin is initialized, evaluators are available through the assertion engine:

```
engine := assertion.NewDefaultEngine()
userflow.RegisterEvaluators(engine)

// Evaluate a status code assertion
def := assertion.Definition{Value: 200}
passed, msg := engine.Evaluate("status_code", def, 200)
// passed=true, msg="status code is 200"

// Evaluate a test failure count
def = assertion.Definition{}
passed, msg = engine.Evaluate("all_tests_pass", def, 0)
// passed=true, msg="all tests passed (0 failures)"

// Evaluate a duration check
def = assertion.Definition{Value: 5000}
passed, msg = engine.Evaluate("within_duration", def, 3200)
// passed=true, msg="duration 3200ms within limit 5000ms"
```

Summary Table

Name	Input	def.Value	Pass When
build_succeeds	bool	--	true
all_tests_pass	int	--	== 0
lint_passes	bool	--	true
app_launches	bool	--	true
app_stable	bool	--	true
status_code	int	int	actual == expected
response_contains	string	string	Contains(actual, expected)
response_not_empty	string/ []byte	--	len > 0
json_field_equals	any	any	Sprintf match
screenshot_exists	[]byte	--	len > 0
flow_completes	bool	--	true
within_duration	int (ms)	int (ms)	actual <= limit